

To capture the specific statistics mentioned in last example — differentiating between **overall simulation time** versus the **active workload window** (ramp-up to ramp-down) — the best architectural approach is to embed an internal telemetry struct inside **ProgrammableFifo**

Because a FIFO can mathematically only be in a **FULL** state *during* the active workload window (it can't be full before the first push or after the final pop), the number of *cycles spent full* remains the same for both calculations. The difference lies in the **denominator** (Total Simulation Cycles vs. Window Cycles).

Enhanced **ProgrammableFifo** module [also added the specific (**Slots filled: X/Y**) telemetry to the logging statements]:

```
// =====  
// MODULE 2: PROGRAMMABLE SIZE FIFO CHANNEL (WITH TELEMTRY)  
// =====  
class ProgrammableFifo : public sc_core::sc_module {  
public:  
    tlm_utils::simple_target_socket<ProgrammableFifo, 32, CA_Traits> target_socket;  
    tlm_utils::simple_initiator_socket<ProgrammableFifo, 32, CA_Traits> init_socket;  
    sc_core::sc_in<bool> clk;  
  
    SC_HAS_PROCESS(ProgrammableFifo);  
  
    ProgrammableFifo(sc_core::sc_module_name name, size_t fifo_size = 4)  
        : sc_core::sc_module(name), target_socket("target_socket"), init_socket("init_socket"),  
m_max_size(fifo_size)  
    {  
        target_socket.register_nb_transport_fw(this, &ProgrammableFifo::nb_transport_fw);  
        init_socket.register_nb_transport_bw(this, &ProgrammableFifo::nb_transport_bw);  
  
        SC_METHOD(drain_cycle);  
        sensitive << clk.pos();  
        dont_initialize();  
    }  
  
    // Standard SystemC callback invoked automatically at the end of sc_start()
```

```

void end_of_simulation() override {
    uint64_t window_total_cycles = stats.window_end_cycle - stats.window_start_cycle;

    // Calculate Utilization Percentages
    double overall_percent = (stats.current_cycle > 0) ?
        (static_cast<double>(stats.cycles_full) / stats.current_cycle * 100.0) : 0.0;

    double window_percent = (window_total_cycles > 0) ?
        (static_cast<double>(stats.cycles_full) / window_total_cycles * 100.0) : 0.0;

    std::cout << "\n===== \n";
    std::cout << "    MODULE TELEMETRY: " << name() << "\n";
    std::cout << "===== \n";
    std::cout << " Metrics | Value \n";
    std::cout << "----- \n";
    std::cout << " Total Items Processed | Pushed: " << stats.items_pushed << " / Popped: " <<
stats.items_popped << "\n";
    std::cout << " Full-State Transitions | " << stats.transitions_to_full << " times the FIFO hit max
capacity \n";
    std::cout << " Total Cycles Evaluated | " << stats.current_cycle << " cycles (Simulation Duration) \n";
    std::cout << " Active Workload Window | " << window_total_cycles << " cycles (" << stats.first_push_time
<< " -> " << stats.last_pop_time << ") \n";
    std::cout << " Cycles spent FULL | " << stats.cycles_full << " cycles \n";
    std::cout << "----- \n";
    std::cout << " % FULL (Overall) | " << std::fixed << std::setprecision(2) << overall_percent << "
% \n";
    std::cout << " % FULL (Active Window) | " << std::fixed << std::setprecision(2) << window_percent << "
% \n";
    std::cout << "===== \n";
}

private:
    size_t m_max_size;
    std::queue<Instruction> m_storage;
    bool m_upstream_notified_full = false;

```

```

bool m_sink_stalled = false;

// --- TELEMETRY TRACKER ---
struct Telemetry {
    uint64_t current_cycle = 0;          // Absolute simulation cycle count
    uint64_t cycles_full = 0;           // How many cycles the FIFO spent in a full state
    uint64_t transitions_to_full = 0;    // How many discrete times it hit the max limit

    uint64_t items_pushed = 0;
    uint64_t items_popped = 0;

    uint64_t window_start_cycle = 0;
    uint64_t window_end_cycle = 0;

    sc_core::sc_time first_push_time = sc_core::SC_ZERO_TIME;
    sc_core::sc_time last_pop_time = sc_core::SC_ZERO_TIME;

    bool is_currently_full = false;
} stats;

tlm::tlm_sync_enum nb_transport_fw(std::optional<Instruction>& tx, tlm::tlm_phase& phase, sc_core::sc_time&
delay) {
    if (phase == tlm::BEGIN_REQ && tx.has_value()) {
        if (m_storage.size() >= m_max_size) {
            return tlm::TLM_REJECTED;
        }

        // Stat tracking: Identify workload ramp-up start
        if (stats.items_pushed == 0) {
            stats.window_start_cycle = stats.current_cycle;
            stats.first_push_time = sc_core::sc_time_stamp();
        }

        m_storage.push(*tx);
        stats.items_pushed++;
    }
}

```

```

        std::cout << "                [FIFO] Accepted: " << tx->op << " (Slots filled: " << m_storage.size() <<
"/" << m_max_size << ")\n";

        if (m_storage.size() >= m_max_size) {
            // Stat tracking: Count discrete transitions to full
            if (!stats.is_currently_full) {
                stats.transitions_to_full++;
                stats.is_currently_full = true;
            }
            phase = custom_phase::BUFFER_FULL;
            m_upstream_notified_full = true;
            return tlm::TLM_UPDATED;
        }
        return tlm::TLM_COMPLETED;
    }
    return tlm::TLM_ACCEPTED;
}

void drain_cycle() {
    // --- TICK TELEMETRY ---
    stats.current_cycle++;
    if (m_storage.size() >= m_max_size) {
        stats.cycles_full++;
    }

    if (m_storage.empty()) return;

    if (m_sink_stalled) {
        // added slot count
        std::cout << "                [FIFO] Blocked. Downstream Execution Unit is busy. (Slots filled: " <<
m_storage.size() << "/" << m_max_size << ")\n";
        return;
    }

    // Packaged item cleanly back inside an optional wrapper for delivery

```

```

    std::optional<Instruction> tx_out = m_storage.front();
    //just changing this to tx_ref (for efficiency/perf reason) do NOT work like the case in module #1 above
    //as the storage here is queue (above is vector) and
    //m_storage.pop() destroys the element at the front and frees or moves its memory
    //std:vector needs to be used again but this case extra logic would then be needed
    tlm::tlm_phase phase = tlm::BEGIN_REQ;
    sc_core::sc_time delay = sc_core::SC_ZERO_TIME;

    std::cout << "          [FIFO] Attempting to dispatch " << tx_out->op << " to execution sink\n";
    tlm::tlm_sync_enum status = init_socket->nb_transport_fw(tx_out, phase, delay);

    if (status == tlm::TLM_UPDATED && phase == custom_phase::BUFFER_FULL) {
        std::cout << "          [FIFO] Destination rejected/stalled. Halting dispatch.\n";
        m_sink_stalled = true;
    } else {
        m_storage.pop();

        // Stat tracking: continuously update the end of the workload window
        stats.items_popped++;
        stats.window_end_cycle = stats.current_cycle;
        stats.last_pop_time = sc_core::sc_time_stamp();
        stats.is_currently_full = false; // Just popped, no longer full

        if (m_upstream_notified_full && m_storage.size() < m_max_size) {
            m_upstream_notified_full = false;
            std::optional<Instruction> empty_signal = std::nullopt;
            tlm::tlm_phase release_phase = custom_phase::BUFFER_FREE;
            target_socket->nb_transport_bw(empty_signal, release_phase, delay);
        }
    }
}

    tlm::tlm_sync_enum nb_transport_bw(std::optional<Instruction>& tx, tlm::tlm_phase& phase, sc_core::sc_time&
delay) {
    if (phase == custom_phase::BUFFER_FREE) {

```

```

        // added slot count
        std::cout << "                [FIFO] Received execution unit clearance. Resuming next cycle. (Slots
filled: " << m_storage.size() << "/" << m_max_size << ")\n";
        m_sink_stalled = false;
        return tlm::TLM_COMPLETED;
    }
    return tlm::TLM_ACCEPTED;
}
};

```

How the Statistics are Captured

- **Deterministic Counting:** The `drain_cycle` runs exactly once per `clk.pos()`. By placing `stats.cycles_full++` at the very top of `drain_cycle`, it guarantees we safely evaluate the state of the FIFO at the clock edge *before* any pops occur. This prevents Delta-Cycle racing.
- **The Window Logic:** SystemC does not natively know when your custom "workload" ends. By capturing `sc_time_stamp()` and `stats.current_cycle` every time an item is **pushed** (if it's the 1st) or **popped** (continuously overwriting the 'last' state), the exact ramp-up / ramp-down window is locked in.
- **Transitions vs Cycles:** It records both how many discrete times the FIFO hit its max cap (`transitions_to_full`), and the sheer number of clock cycles it spent jammed up (`cycles_full`).

Expected Telemetry Output

When the `sc_start(160, sc_core::SC_NS);` completes in the testbench, SystemC will auto-trigger `end_of_simulation()` yielding this appended output:

```

=====
MODULE TELEMETRY: internal_fifo
=====
Metrics                | Value
-----
Total Items Processed   | Pushed: 6 / Popped: 6
Full-State Transitions  | 2 times the FIFO hit max capacity
Total Cycles Evaluated  | 16 cycles (Simulation Duration)
Active Workload Window  | 8 cycles (60 ns -> 140 ns)

```

Cycles spent FULL | 3 cycles

% FULL (Overall) | 18.75 %

% FULL (Active Window) | 37.50 %
=====